OPERATING SYSTEMS

ORANGE PROBLEM

NAME – PURAV N KHINCHA

SRN – PES1UG24CS349

SECTION – 4F

<u>SCREENSHOTS –</u>

1A)

```
vboxuser@UbuntuVM:~/os-orange$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
vboxuser@UbuntuVM:~/os-orange$
```

1B)

```
vboxuser@UbuntuVM:~/os-orange$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
vboxuser@UbuntuVM:~/os-orange$
```

2A)

```
vboxuser@UbuntuVM:~/os-orange$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
vboxuser@UbuntuVM:~/os-orange$
```

2B)

```
vboxuser@UbuntuVM:~/os-orange$ xxd $(find .pes/objects -type f | head -1) | head -20
00000000: 7472 6565 2034 3900 3130 3036 3434 2066  tree 49.100644 f
00000010: 696c 6531 2e74 7874 002c f8d8 3d9e e295  ile1.txt.,..=...
00000020: 43b3 4a87 7274 21fd ecb7 e3f3 a183 d337  C.J.rt!........7
00000030: 6390 25de 576d b9eb b4                   c.%.Wm...
vboxuser@UbuntuVM:~/os-orange$
```

3A)

```
vboxuser@UbuntuVM: ~/os-orange
vboxuser@UbuntuVM:~/os-orange$ echo "hello" > hello.txt
vboxuser@UbuntuVM:~/os-orange$ echo "world" > world.txt
vboxuser@UbuntuVM:~/os-orange$ ./pes add hello.txt world.txt
vboxuser@UbuntuVM:~/os-orange$ ./pes status
Staged changes:
  staged:     hello.txt
  staged:     world.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.c
  untracked:  file1.txt
  untracked:  test_objects.c
  untracked:  pes.h
  untracked:  tree.h
  untracked:  index.h
  untracked:  file2.txt
  untracked:  object.c
  untracked:  index.c
  untracked:  test_objects
  untracked:  tree.c
  untracked:  Makefile
  untracked:  test_tree
  untracked:  commit.h
  untracked:  test_tree.c
  untracked:  README.md
  untracked:  pes.c
  untracked:  test_sequence.sh

vboxuser@UbuntuVM:~/os-orange$
```

3B)

```
vboxuser@UbuntuVM:~/os-orange$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776774532 6 file1.txt
```

4A)

```
vboxuser@UbuntuVM:~/os-orange$ ./pes log
commit cbebeccc2269dc3088aa1c5fbfa9826a9df2ee5ef8d026718053931065bc062a
Author: PES User <pes@localhost>
Date:   1776774888

    third commit

commit ee7d4f7e2dea2826823174f7e4711b770ef1c2dd14ac97e1b502bae280accc7f
Author: PES User <pes@localhost>
Date:   1776774875

    second commit

commit 90f9a84d68398afbf4ad842b64127e3a39b4b9de0475de5c09dc86a65f2cb6d0
Author: PES User <pes@localhost>
Date:   1776774557

    initial commit

vboxuser@UbuntuVM:~/os-orange$
```

4B)

```
vboxuser@UbuntuVM:~/os-orange$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/09/b576c2eb51b78fcf9b8493a0909522a75f8ee86df0e926312deeb5fbe7dfb1
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/90/f9a84d68398afbf4ad842b64127e3a39b4b9de0475de5c09dc86a65f2cb6d0
.pes/objects/cb/ebeccc2269dc3088aa1c5fbfa9826a9df2ee5ef8d026718053931065bc062a
.pes/objects/ee/7d4f7e2dea2826823174f7e4711b770ef1c2dd14ac97e1b502bae280accc7f
.pes/refs/heads/main
```

ANSWERS

Q5.1 — Implementing pes checkout <branch>

To implement pes checkout <branch>, the following files in .pes/ need to change:

1. **.pes/HEAD** — rewritten to ref: refs/heads/<branch> (or the commit hash directly if detached)
2. **.pes/refs/heads/<branch>** — read to find the target commit hash
3. **.pes/index** — rebuilt to reflect the files in the target branch's tree

The steps would be:

1. Read .pes/refs/heads/<branch> to get the target commit hash
2. Read and parse that commit object to get its root tree hash
3. Recursively walk the tree object, and for every blob entry, extract the file content and write it to the working directory

4. Delete any working directory files that exist in the current tree but not in the target tree
5. Rebuild the index to match the target tree's contents
6. Update .pes/HEAD to point to the new branch

What makes it complex: You have to reconcile three states simultaneously — the current index, the working directory, and the target tree. If a file was modified in the working directory but not staged, and the target branch has a different version of that file, you must refuse the checkout to avoid silently discarding the user's work. Handling new untracked files, deleted files, and nested subdirectories all add edge cases.

Q5.2 — Detecting Dirty Working Directory Conflict

Using only the index and object store:

1. For each entry in the current index, stat() the file in the working directory
2. Compare mtime and size against what's stored in the index entry — if they differ, the file is **locally modified**
3. For files that show metadata changes, re-hash the file content and compare against the blob hash in the index — if it differs, the file is **truly dirty**
4. Look up what hash the target branch's tree has for that same path
5. If the working directory version differs from both the current index blob AND the target branch's blob, **refuse checkout** and report a conflict

Files that are dirty but happen to match the target branch's version can be safely checked out (the user's change is already what the target branch has). Files not tracked in the index at all (untracked) are generally left alone unless the target branch creates a file at the same path.

Q5.3 — Detached HEAD and Recovery

What happens when you commit in detached HEAD state: HEAD contains a raw commit hash instead of ref: refs/heads/main. When you make new commits, head_update writes the new commit hash directly into HEAD. These commits are valid objects in the object store and form a real chain — but no branch name points to them. They are "floating" commits.

How to recover: If the user hasn't lost track of the hash (it's still printed on screen or in terminal history), they can create a branch pointing to it:

bash

```
git branch recovery-branch <commit-hash>

git checkout recovery-branch
```

If they've already checked out another branch and lost the hash, Git's **reflog** (.git/logs/HEAD) records every position HEAD has been at, so they can find the dangling commit hash there. In PES-VCS (which has no reflog), recovery would require scanning all objects in .pes/objects/ to find commits not reachable from any branch — essentially a reverse garbage collection.

Q6.1 — Garbage Collection Algorithm

Algorithm (mark-and-sweep):

1. **Mark phase** — Start from every branch ref in .pes/refs/heads/. For each, walk the commit chain following parent pointers. For each commit, add its tree hash to a reachable set, then recursively walk that tree, adding every subtree and blob hash encountered. Use a **hash set** (e.g. a hash table keyed by the 32-byte ObjectID) to track all reachable hashes — O(1) insert and lookup.
2. **Sweep phase** — Walk every file under .pes/objects/. For each file, reconstruct its ObjectID from the path. If it's not in the reachable set, delete it.

Estimate for 100,000 commits + 50 branches:

- Each commit points to 1 root tree; each tree might have ~10 entries on average (subtrees + blobs)
- Assume average tree depth of 3 and branching factor of 5 → roughly 10–50 tree/blob objects per commit
- Total objects to visit: ~100,000 commits × ~20 objects = **~2 million objects** across all 50 branches, with significant overlap (shared blobs/trees counted once due to the hash set deduplication)

Q6.2 — GC Race Condition with Concurrent Commit

The race condition:

1. A commit operation calls tree_from_index() and object_write() to write several blob and tree objects to the store — but hasn't yet written the commit object or updated HEAD
2. GC starts at this exact moment, walks all refs from HEAD, finds these new blobs and trees are not reachable from any commit (because the commit object doesn't exist yet), and **deletes them**
3. The commit operation then tries to write the commit object referencing those now-deleted tree/blob hashes — the repository is now corrupt

How real Git avoids this:

- Git writes a **lock file** (.git/index.lock) during index operations, and uses per-ref lock files during ref updates, so GC can detect an in-progress operation
- Git's GC also has a **grace period** — it never deletes any object created in the last 2 weeks (configurable via gc.pruneExpire), so a new object written mid-commit is safe even if GC runs concurrently
- Additionally, Git GC refuses to run if it detects recent activity on lock files, making the window of danger essentially zero in practice